Experiment: Implementation and Simulation of Distance Vector Routing Protocol Using C

Aim

To implement and simulate the **Distance Vector Routing Protocol** using the **Distance Vector Routing Algorithm (Bellman-Ford Algorithm)** in C to determine the shortest path and routing table for each router in a network.

Definition

The **Distance Vector Routing Protocol (DVR)** is a dynamic routing algorithm in which each router maintains a **routing table (distance vector)** containing the shortest distance to every destination in the network and the next hop required to reach it.

Each router periodically exchanges its routing table with its neighboring routers. Based on the received information, routers update their routing tables using the **Bellman-Ford Equation** until all routers have the shortest paths to every destination.

Description

Distance Vector Routing is widely used in computer networks to determine the best path for forwarding packets. Initially, each router knows only the distance to its directly connected neighbors. During periodic updates, routers exchange their routing tables with neighboring routers.

Whenever a router receives a routing table from a neighbor, it computes a new distance using:

$$D_x(y) = \min_v \{C(x,v) + D_v(y)\}$$

where

- $D_x(y)$ = Distance from router **x** to destination **y**
- $C(x,v)$ = Cost from router **x** to neighbor **v**
- $D_v(y)$ = Distance from neighbor **v** to destination **y**

If the new distance is shorter than the existing one, the routing table is updated.

This process continues until no further updates occur (convergence).

C Program

```
#include<stdio.h>
#define MAX 10
#define INF 999
struct Router
{
   int distance[MAX];
   int from[MAX];
} rt[MAX];
int main()
{
   int cost[MAX][MAX];
   int n;
   int i,j,k;
   int count=0;
   printf("Enter the number of routers: ");
   scanf("%d",&n);
   printf("\nEnter the Cost Matrix:\n");
   for(i=0;i<n;i++)
   {
      for(j=0;j<n;j++)
      {
         scanf("%d",&cost[i][j]);
         if(cost[i][j]==-1)
            cost[i][j]=INF;
         rt[i].distance[j]=cost[i][j];
```

```
      rt[i].from[j]=j;
        }
    }
    do
    {
       count=0;
       for(i=0;i<n;i++)
       {
          for(j=0;j<n;j++)
          {
             for(k=0;k<n;k++)
             {
                if(rt[i].distance[j] >
                  cost[i][k] + rt[k].distance[j])
                {
                   rt[i].distance[j] =
                   cost[i][k] + rt[k].distance[j];
                   rt[i].from[j]=k;
                   count++;
                }
             }
          }
       }
    }
while(count!=0);
    printf("\n=======================================\n");
```

```
    for(i=0;i<n;i++)
    {
        printf("\nRouting Table for Router %d\n",i+1);
        printf("Destination\tNext Hop\tDistance\n");
        for(j=0;j<n;j++)
        {
            printf("%d\t\t%d\t\t%d\n",
                    j+1,
                    rt[i].from[j]+1,
                    rt[i].distance[j]);
        }
    }
    return 0;
}
```

Sample Input

Enter the number of routers:

4

Enter the Cost Matrix:

0 2 7 1

2 0 4 3

7 4 0 2

1 3 2 0

Sample Output

==========

Routing Table for Router 1

Destination	Next Hop	Distance
1	1	0
2	2	2
3	4	3
4	4	1

Routing Table for Router 2

Destination	Next Hop	Distance
1	1	2
2	2	0
3	3	4
4	4	3

Routing Table for Router 3

Destination	Next Hop	Distance
1	4	3
2	2	4
3	3	0
4	4	2

Routing Table for Router 4

Destination	Next Hop	Distance
1	1	1
2	2	3
3	3	2
4	4	0

Result:

The **Distance Vector Routing Protocol** was successfully implemented and simulated using the **Bellman-Ford algorithm** in C. The program correctly computed the shortest paths between routers, updated the routing tables through iterative exchanges, and displayed the final routing table with the minimum distance and next-hop information for all destinations.